

OMegaETH: Data Availability (DA) Cost-Efficient Layer 2 Transaction Ordering Optimization

Kang Taejeong, Kwon Jongheon, Park
Sanghyun, Yim Sehun

1.2 모듈러 블록체인과 Layer 2

Execution

→ 트랜잭션 실행 (EVM, state transition)

Consensus

→ 어떤 블록이 정답인지 합의 (PoW, PoS)

Data Availability (DA)

→ 데이터가 실제로 네트워크에 퍼져있는지 보장

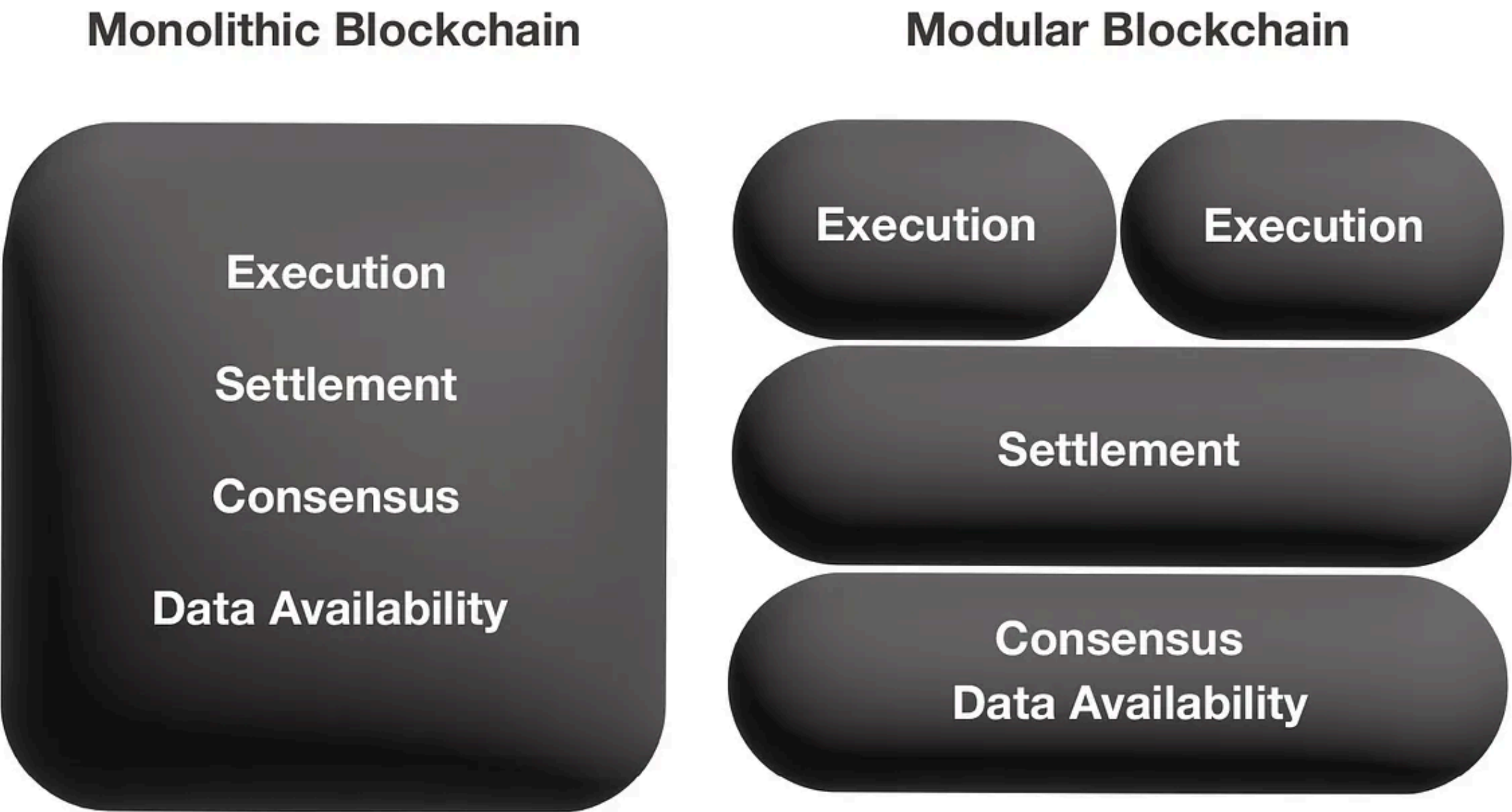
Settlement

→ 최종 확정 (finality)

The Old Model: Monolithic Blockchain

:: FOUR PILLARS

Difference Between Monolithic and Modular Framework



- EIP-4844 (Dencun 업그레이드) 도입: L2 가스비 절감을 위해 ‘Blob’ 도입
- 새로운 제약 조건 발생: 모든 Blob은 128KB 고정 크기
- 구조적 모순 (비용 불균형):
 - 사용자는 자신이 사용한 데이터(연산) 크기만큼 수수료 지불
 - L2 운영자는 빈 공간이 있어도 무조건 128KB 도매가 지불

Table 1. Rollup Blob Utilization and Performance: Blocks 19426589 to 20611514

Rollup	Blob Count	Average Size (KB)	Total Size (GB)	DA Quality
Base	690356	126.51 (98.84%)	83.293	0.309
Arbitrum	375014	127.52 (99.63%)	45.607	0.288
Taiko	328805	30.70 (23.99%)	9.628	0.592
OP Mainnet	265469	126.71 (98.99%)	32.080	0.239
Scroll	139199	105.35 (82.30%)	13.985	0.332
Blast	106033	115.03 (89.87%)	11.632	0.255
Linea	93604	117.64 (91.90%)	10.501	0.237
StarkNet	62361	127.98 (99.98%)	7.611	0.242
zkSync Era	62149	128.00 (99.99%)	7.586	0.215
Paradex	44300	127.99 (99.99%)	5.407	0.234
Metal	31382	0.18 (0.14%)	0.005	0.220
Zircuit	27821	6.04 (4.72%)	0.160	0.275
Kroma	24193	59.65 (46.60%)	1.376	0.208
Zora	23592	126.58 (98.89%)	2.848	0.161
Mode	22585	126.56 (98.88%)	2.726	0.159
Rari	19583	31.08 (24.28%)	0.581	0.214
Optopia	7851	16.19 (12.65%)	0.121	0.179
Boba Network	6236	3.29 (2.57%)	0.020	0.166
Debank Chain	3358	113.84 (88.94%)	0.365	0.162
Camp Network	3292	98.39 (76.87%)	0.309	0.136
Nal	2998	0.21 (0.17%)	0.001	0.168
Mint	2515	90.89 (71.01%)	0.218	0.140
Lambda	2336	103.80 (81.09%)	0.231	0.143
Lumio	1390	0.43 (0.33%)	0.001	0.137
Parallel	1348	119.29 (93.20%)	0.153	0.113
XGA	1031	98.20 (76.72%)	0.097	0.142
Lisk	811	8.04 (6.28%)	0.006	0.126
Kinto	204	13.20 (10.31%)	0.003	0.124

- 단순 선착순(FCFS) 스케줄링의 한계: 트랜잭션이 모일 때까지 무작정 기다릴 수 없음
- Latency 딜레마: 대기 시간(Timeout) 초과 시, 빈 블록 강제 제출
- 실제 현업 데이터 양극화 현상:
- Base/Arbitrum(99% 충전)과 극소형 롤업(0.1%)을 제외한, 트래픽이 존재하는 중견 네트워크의 낭비 심각.
- Linea(91%), Scroll(82%), Kroma(46%) 등은 트랜잭션이 꾸준히 발생함에도 FCFS의 한계로 인해 지속적으로 10~20% 이상의 블록 공간(비용)을 낭비.
 - 결과적으로 전체 네트워크의 평균 공간 효율은 70~80%에 불과

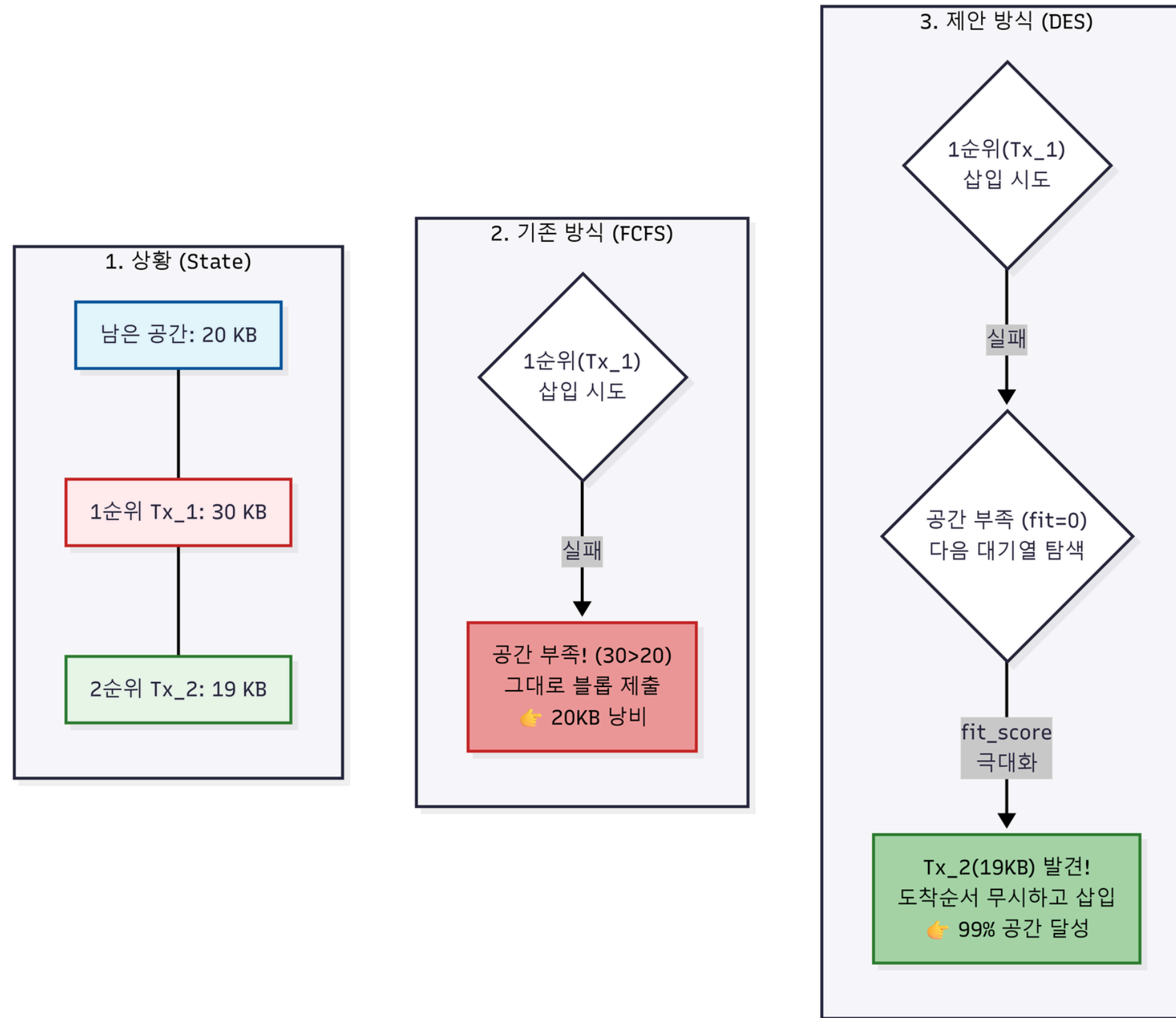
- 거시적(Macro) 접근 - Blob Sharing의 한계:
 - 타 롤업 네트워크 간의 데이터 병합. 초소형 롤업(예: Metal)에게는 유효하나, 경쟁 네트워크 간의 복잡한 프로토콜 협력과 인센티브 설계가 필요하여 단기 적용이 불가능.
- 최신 스케줄러 (예: TimeBoost):
 - 외부 협력 없이, 단일 롤업의 시퀀서 내부 스케줄링 최적화만으로 Scroll, Linea 같은 중견 네트워크의 밀도를 극대화하는 현실적 타겟팅.
 - 우선순위(Tip)와 도착 시간만 고려.
 - 데이터 압축률이나 블록의 '빈 공간 맞춤(Fit)'은 전혀 고려하지 않음.
- 결론: 단일 시퀀서 내부에서 작동하는 미시적(Micro) 밀도 최적화 로직의 부재.

- DA Efficiency Score (DES) 모델 제안
- 단순 FCFS를 넘어, 밀도를 높이는 테트리스 방식의 오더링

공식: $DES = (\alpha * wait_score) + (\beta * compress_score) + (\gamma * fit_score)$

- wait_score: 영원히 처리되지 않는 기아(Starvation) 방지 가중치
- compress_score: 스마트 컨트랙트 등 트랜잭션 타입별 통계적 압축률 우대
- fit_score: 현재 블록의 남은 빈 공간에 정확히 들어맞는지(Knapsack) 평가
 - 예시 (Knapsack/테트리스 효과): 남은 블록 공간이 20KB일 때, 다음 차례가 30KB 트랜잭션이라면 FCFS는 블록을 그대로 닫아버려 20KB를 버림.
 - 반면, DES는 Mempool을 깊게 탐색하여 19KB 트랜잭션을 찾아 빈틈을 완벽히 채움(Fit).

해결 방안 - DES 스케줄링 알고리즘



- 프라이빗 Devnet 구축 (Kurtosis & Docker)
 - 실제 Ethereum Deneb 환경 완벽 재현.
- L2 Custom Sequencer 자체 개발 (Rust)
 - 베이스라인(FCFS)과 제안 모델(DES)을 모두 구현하여 스위칭/비교.
- 현실적 워크로드 시나리오 (Dataset Generator)
 - a. Small-heavy (단순 송금 중심)
 - b. Large-heavy (복잡한 컨트랙트 중심)
 - c. Mixed (일반 범용 혼합형)
- Latency 제약 조건 반영 (batch_timeout_ms)
 - 무한정 대기하는 것을 방지하기 위해, 트랜잭션 대기 시간 임계치 도달 시 블롭 공간이 비어있어도 강제 제출하도록 시뮬레이션 환경을 구성하여 현실성 입증.



- 목표: 비용 절감과 Latency 억제의 최적 균형점 도출
- 비용 관점 (Cost):
 - - 낭비되는 블록 개수 15~30% 감소
 - - 평균 블록 충전율(Fill rate) 70% -> 85% 이상 개선
- 성능 관점 (Latency):
 - - 평균 대기시간 (Avg Latency)
 - - P95 및 Max Latency 변동 억제 측정

- 완료 사항: Devnet 인프라 구축 및 시퀀서 핵심 로직(FCFS/DES) 연동 완료.
- 단기 목표 (High): EIP-4844 Blob Sender 로직 완료, Dataset Generator 파이프라인 구축.
- 중기 목표 (Medium): 3가지 워크로드별 비교 실험 및 최적 파라미터 (α , β , γ) 스위핑 탐색.
- 최종 목표 (Low): 비용 vs Latency 시각화, 지표 분석 및 논문/전시 자료 집필.

팀원	주요 역할
강태정	프로젝트 총괄, 인프라 구축(Devnet), Custom Sequencer 코어 구현 (Rust)
권정현	DES 이론 모델 수립, 파라미터(α , β , γ) 최적화 및 Knapsack 메커니즘 분석
박상현	선행 연구(TimeBoost 등) 분석, 워크로드 기획, 트랜잭션 데이터 파이프라인 구축
임세훈	실험 결과 통계 분석, Latency 분석 및 최종 성과 도출

Q&A